

# Topics

## Page No

|                                              |    |
|----------------------------------------------|----|
| (1) Validation in C#                         | 1  |
| (2) Strings                                  | 3  |
| (3) String Builder                           | 5  |
| (4) difference b/w String And SB             | 6  |
| (5) State Management technique.              | 7  |
| (6) Exception Handling                       | 10 |
| (7) Delegates                                | 12 |
| (8) Extension Method                         | 14 |
| (9) Anonymous function                       | 15 |
| (10) Serialization, Deserialization          | 16 |
| (11) Garbage collector                       | 17 |
| (12) Structure,                              | 18 |
| (13) Enum                                    | 19 |
| (14) Local and Global Variable               | 20 |
| (15) Link                                    |    |
| (16) OOPS, <del>Array List</del> , Array     | 21 |
| (17) Method hiding                           | 25 |
| (18) Static class                            | 34 |
| (19) Sealed class                            | 35 |
| (20) difference b/w Ref / out                | 35 |
| (21) " " Value type / Reference type         | 37 |
| (22) " " class and structure                 | 37 |
| (23) " " title of and get type.              | 37 |
| (24) " " Parse and Convert.                  | 38 |
| (25) " " dispose and finalize.               | 38 |
| (26) " " Response.Redirect & Server.Transfer | 39 |
| (27) Indexer                                 | 39 |
| (28) Partial class                           | 40 |
| (29) Coupler                                 | 41 |
| (30) Boxing and Unboxing                     | 42 |
| (31) Dependency Injection                    | 43 |
| (32) Collection                              | 44 |

(Array List, Array, Queue, Stack, Hash Table, Dictionary)

- (31) Role based Management
- (34) Singleton
- (35) ViewBag, ViewModels, TempData
- (36) Filter
- (35) Generics
- (36) Routing
- (37) Partial View
- (38) Access Modifier
- (39) Convert.ToString() vs ToString()
- (40) IEnumerable, IQueryable
- (41) View Engine
- (42) Query String
- (43) diff. b/w XSS and SQL Injection

(59)



# Validation in .Net

## Why Need

It is important part of any web application. User input's must be always be validated before sending across different layers of the application.

Validation Check करती है कि User की भेजा गया Data Valid Data है या नहीं।

## Types of Validation

→ Client Side Validation

→ Server Side Validation

### • Client Side Validation:

It is good but we have to be dependent on browser and scripting language support.

### Advantages

- the main advantage is that it prevents a page from being posted to the server until the client validation is executed successfully.

यदि वह सफल तरीके से सफल रूप से  
validate नहीं है तो वह सफल रूप से  
server तक नहीं जाएगा।

# Server Side Validation

It work on both end: first it will work on client validation and then on server validation.

What is Validation Control?

⇒ Validation Controls are responsible to validate data of an input control.

Asp.Net 4.0 contains the 6 Types of Validation Control.

- (1) Required Field Validation Control
- (2) Compare Validator Control.
- (3) Range Validator Control
- (4) Regular Expression validator Control
- (5) Custom Validator Control
- (6) Validation Summary.

## Required Field Control

- Check either a control is empty or not.

જો કંટ્રોલ એ થ્રોવ ફી ટેક્સ્ટ બોક્સ એ એમ્પી  
(without fill any data) data એકે રજા રાખે છે.

અમે બિન ફોર્મ એ સબમિટ બટન એ થ્રોવ સબમિટ  
બોટો 1 નો ચેક કરીએ એ એકે ટેક્સ્ટ બોક્સ  
રજા રાખે એ એકે એકે રજા રાખે છે.  
1 msg print કરીએ.



## Compare Validator Control

ये को Control (Text box) में जोड़ा है  
Compare करता है।

It must be used for "password" and  
confirm password.

## Range Validator Control

Range Validator Control Check to see if  
a control value is within a valid  
Range or not.

मान लो कि Text box (Control) है जिसमें हम  
Phon No. input करते हैं अब Phon No. में  
1 से 10 तक का होना है जो हमें देना

Text box में 1 से 10 तक Range Set कर  
देते। जिससे User Text box में 10 से कम  
कोई value नहीं रख सके।

हमारा जो Range 0-9 है Set कर दें।

## Important Attribute are

Maximum value, minimum value and Type.

Ex. maximum Value = "100" minimum Value = "10"  
Value = "integer" >

## Regular Expression Validator

21E Email के लिए Apply Input Validator

these expressions can be Phone Numbers, email address, zip code etc.

## Custom Validation

31A 31A पाने है 31A की Validate Code  
1PKAAT | 31A के लिए custom Validator Control  
use फंक्शन

## Validation Summary

This Control will collect all the error message of all non valid Controls and put them in a tidy list.





ये जो Symbol में दले जाते हैं उन्हें  
O/P में Print नहीं देना है

|              |          |
|--------------|----------|
| (i) \ "AA \" | O/P "AA" |
| ' AA '       | O/P 'AA' |
| AA           | O/P AA   |
| " AA "       | O/P AA   |

t = true

String a = "one \n two \n three";    O/P    one  
two  
three

## C# String Functions

Clone ()      make clone of String.

CompareTo ()      Compare two strings and return Integer value  
0 = true, 1 = false

EndsWith ()      Check specific character is last or not.

GetHashCode ()      Return Hash value, ;

Replace ()

Split ()

Remove ()

Substring ()

ToLower ()

ToUpper ()

IndexOf ()



P-4

(4)

```
String str = "Dot Net Tricks Skill Challenge";  
String[] strArray = str.split(' ');  
① for (int i = 0; i < strArray.length; i++)  
    {  
        Console.WriteLine("{0}", strArray[i]);  
    }
```

O/P = Dot  
Net  
Tricks  
Skill  
Challenge.

```
String str2 = str + str;  
String str1 = str.concat(str, ", you");  
C.W (str1);  
② O/P => Dot Net Tricks Skill Challenge, you.
```

```
String str2 = str.IndexOf("Skill");  
C.W (str2);  
C.W (str2);  
③ O/P = 15
```

```
String str3 = str.Replace("Tricks", "Tricks's");  
C.W (str3);  
④ O/P = Dot net Tricks's Skill Challenge.
```

## ToLower

String lower = str.ToLower();

Console.WriteLine(lower);

O/P dot net tricks skill challenge.

## ToUpper

String upper = ~~str~~ str.ToUpper();

Console.WriteLine(upper);

O/P ⇒ DOT NET TRICKS SKILL CHALLENGE.

## Substring

String substr = str.Substring(0, 14);

Console.WriteLine(substr);

O/P = Dot net tricks.

String is object. Heap Memory में  
असता है। जो value में है & वह  
change हो सकेगा। memory block में  
असता है।



# String Builder

It is mutable.

जब भी हम String Builder को change करते हैं उसे  
new memory block में ले जाते हैं। इससे यह पक्का  
है कि String Builder Append करने पर कोई भी  
पुराना String नहीं बदलता।

String Builder शुरू में default 16 character

की capacity होता है।

जिससे अगर हम String Builder में

जैसे

```
Stringbuilder sb = new Stringbuilder(50);
```

तो इसकी capacity 50 character है।

## Append and AppendLine

Use Append method of String builder to add  
or append a String.

E.g Stringbuilder sb = new Stringbuilder("Adil");

```
sb.Append("xyz");
```

```
CLW (sb);
```

O/P Adilxyz

Append Line → Append the String new line.

E.g

```
Stringbuilder sb = new Stringbuilder("AB");
```

```
sb.Append("Devilien"); sb.AppendLine("ABC");
```

```
sb.Append("Batman");
```

```
CLW (sb);
```

O/P

AB Devilien ABC

Batman

## Insert()

String Builder sb = new StringBuilder("Hello world");

sb.insert(5, "c#");

cout << sb;

OP = Hello c# world.

## Remove()

String Builder sb = new StringBuilder("Hello world");

sb.remove(6, 5);

OP = Hello.

Remove(6, 5);

Start

↓  
1st char to  
Remove del

## Replace()

• ToString() → Get string from String Builder.

## Points

• String Builder (S.B) is faster.

• ~~String~~ Append()

• ToString()



(6)

## String

- Immutable
- ~~Reference type~~
- ~~Heap memory~~ ~~it~~  
~~store data~~

• System.String

- Always create new object

• Slower

- Concatenation is used to combine two strings

## String Builder

- mutable.

• ~~Value type~~

- ~~Stack memory~~ ~~it~~ ~~store~~

• ~~System.Text~~  
System.StringBuilder

- No need to create new object. it will update string at one place.

• Faster.

- Append method is used

# STATE MANAGEMENT TECHNIQUE

जब मैं Submit button पर Click करूँ  
तो वह data जो username or  
password के control या (text box) में  
स्थिति में है Lost हो जाती है।  
जबकि Page Reload होने के कारण data Lost  
हो जाती है।  
इसलिए हम State Management Technique use  
करते हैं।  
यह दो Type में होता है।

- ① Client Side
- ② Server Side

## Client Side State Management Technique

- ① Hidden field
- ② View State
- ③ Cookies
- ④ Query String

### Hidden field

It is a control provide by ASP.net.  
which is use to store small Amount  
of data on the client. it store one  
value for the variable.

```
<asp:HiddenField ID="Hidden-Field" runat="server"/>
```



## VIEW STATE

Can store any kind of data because it is an object type.

We can transfer data with only same (single) Page.

viewstate ["AB"]

- HTTP PROTOCOL is stateless

Stateless meaning  $\Rightarrow$  it does not store any data

## COOKIES

- We can store the information for later use.
- Cookies are small files that are created in the web browser memory.
- Cookies is a small piece of information stored on the client machine.

```
Ex. HTTPCookie cookie = new HTTPCookie("user");  
cookie["username"] = textbox1.text  
Response.Cookies.Add(cookie);
```

Location : Setting  $\Rightarrow$  Advance  $\Rightarrow$

Content Setting  $\Rightarrow$  Cookies  $\Rightarrow$  See all  
Cookies site data

Now Search "localhost" you can  
see "localhost" click it  
then show the cookie file  
"user".

username

Submit

## Query String

It is Collection of Characters input or group of keywords that send Request to the ~~Browser~~ web Server.

- A Query String is helpful when we want to transfer a value from one Page to another Page.

Id User id

जब हम Submit button पर click करें

तो वह हमारे Id वाले ~~data~~ पर URL  
में show करेगा

Submit

URL में ~~data~~ next Page में /

Ex

Response.Redirect ("nextPage.aspx?id=" + TextBox1.Text);

यहाँ Query String variable defined

? में Separat करेंगे।

जब हम Submit करेंगे तो Id की URL में ~~data~~   
next Page पर Receive करेंगे।

TextBox2.Text = Request.QueryString[0];

→ index number.

जहाँ Request आती है।

जब हम Request में TextBox2 में set कर देंगे  
तो हमें show हो जायेगा।

यहाँ हम उस Data को ~~data~~ में ~~data~~ करेंगे।



3) Multiple data store {1st & 4 use 2nd}

i.e

```
Response.Redirect("nextPage.aspx?id=" + IdTextBox.  
Text + "&name" + NameTextBox.Text + "&age" +  
AgeTextBox.Text);
```

3) Next page to get 2nd

```
TextBox2.Text = Request.QueryString[0];
```

```
TextBox3.Text = Request.QueryString[1];
```

```
TextBox4.Text = " " [2];
```

Query String URL to throw data into next.

Query String

- ① It is Client side S.M.T
- ② Query String data is Page specific (can be accessed in that page only)
- ③ Query String data is visible can be seen in URL.

Session

- ① It is Server side S.M.T.
- ② Session data can be accessed throughout the session.
- ③ Session data is not visible to user.

# SERVER SIDE

(9)

Session :- { Single user + multiple pages }

Store and access value from one page to multiple pages.

We can transfer data one page to multiple pages with single user.

i.e. `Session["User"] = UserTextBox.Text`  
`Response.Redirect("WebForm2.aspx");`

अब यह स web form 2 पर गेट गेट डिजाइनेड फॉर्म है  
इस में जो data को दूसरे page पर collect  
होगा।

## Application :-

यह हम data को multiple pages पर  
multiple user के साथ communicate करे सके हैं  
जो में इसने करे में जो data को other  
New web browser open करे check करे  
करे पर Access हो जायेगा। लेकिन यह session में  
Access नहीं होगा।

i.e. `Application["User"] = UserTextBox.Text`



## Exception Handling

A exception is an event, which occurs during the execution of program that disrupts the normal flow of the program's instructions.

Exception जब भी occur होता है। एप्लीकेशन प्रोग्राम को crash न हो जाए। ठीक हो जाए।

2. Number 0 (zero) is divided by zero exception raised.

(# क इन्फेक्शन की class है जो Exception class descend है जो ~~क~~ लाइ 2/10 पर होता

Q. 1. What is the difference between a class and an object? 2/5

Catch mechanism catch  $\frac{5}{2\sqrt{11}}$  / default

3. default catch mechanism  
 catch block is not present in the application it  
 terminate the program. 3 step follow

Step 1  $\rightarrow$  Application terminate & kill

Step 2 → It ugly all msg print after

Step 2  $\rightarrow$  In line 42 exception not  $\nexists$   
So it's a statement execute after that

1/0

2nd Case it divide by zero exception in 1st

Step 1: 10 steps follow

उसके बाद के लिए खुद से एक ~~क~~

इस code की रकबत की हम user-friendly message print कर सकते हैं exception को और कम कर सकते statement की run कर सकते हैं

try ~~1/0~~

{ 1/0 → ~~error~~

}

Catch (DivideByZeroException ex)

{ C.W("You can not divide Number by 0")  
}

जहाँ (जिस statement में exception आने का chance होगा उसमें हम try Block में रखेंगे।)

try { 1/0; यह जिस class में object throw करेंगे }

Catch (Exception ex यह जहाँ catch कर लेंगे।)

{

}

Statement 1;

Statement 2;



# TRY Catch with Finally Block

- C# finally Block always execute whether exception is occur or not.
- Finally Block में एक Important code रखें जो कि हर सिल में चलना है। जैसे Close connection

```
Try { 1/0; }
```

```
Catch (Exception ex)
```

```
{ Console.WriteLine(ex.Message);  
}
```

~~Step 2~~

```
Finally {
```

```
Console.WriteLine("Final Block always execute");  
}
```

## FOUR WAYS OF EXCEPTION IN C#

- Default throw Default Catch
- " " " Our Catch (Try & Catch)
- Our throw ~~Our~~ Default Catch
- " " " Our Catch

throw keyword use करें।

## OUR ~~default~~ default catch

throw → इस से exception generate करत है  
throw keyword use करत है

बहुत case ऐसे होते हैं जहाँ ~~code~~ code में error होता है  
यहाँ इस से exception generate करत है  
कहाँ कहीं इस exception generate करत है

Ex if (age >= 18)

{  
    c.w ("you can vote");

}

else {

    throw new Exception("Can not vote");

इस से generate करत है exception

}

## OUR throw OUR catch

Ex try { if (age >= 18)

    { c.w ("can vote");

    }

    else { throw new Exception("No vote");

    }

}

catch (Exception ex)

{ c.w (ex.message);

}



# Delegates

- It is also called function pointer.
- Delegate is a type which holds a method's reference in an object.

- It is reference type.

- It has No Implementation means No Body.

- We can use Invoke() methods with it.

यहाँ एक function बनाया है जो सीधे direct call में ही काम करेगा।

```
public void Addition(int a, int b)
```

```
{ int c = a + b;
```

```
    Console.WriteLine(c);
```

```
}
```

जो मैन में जो delegate बनाया है जो इस function को point करे उसे मैन में।

जो है delegate बनाया है उसे class में और

Namespace में कर दिया।

namespace delgen

```
{
```

```
    public delegate void Calculation(int a, int b)
```

Return type और फंक्शन का method का है।

Same signature.

class Program

```
{
```

```
    public static void Addition(int a, int b)
```

```
{ int c = a + b;
```

```
    Console.WriteLine(c);
```

3rd delegate to object & number

Static void main (String args)

{ Program P = new Program();

Calculation Obj = new Calculation(~~Calculation~~  
P ~~Program~~ Addition);

Obj.Invoke (2, 5);

(.Readline());

O/P = 7

3rd delegate to use जो इन्हें चूंकि है  
direct जो कर सकते हैं

साथ चला use कर है कि एक method  
को Indirectly way में call करना है

method को encapsulate करने है

जो जोड़ एक से बढ़ा function है

होते Same Parameter की Substraction, Multiplication  
division method को जोड़ें जो call कर सकते हैं

जो से बढ़ा function जिस Signature, Return  
type Same है जो जोड़ जोड़ जोड़ जोड़  
delegate जोड़ जोड़ जोड़ जोड़ जोड़ जोड़ जोड़  
को जोड़ जोड़



# TYPES OF DELEGATES

→ MULTIPLE Delegates

⇒ Single Cast "

⇒ MULTI CAST "

## MULTIPLE Delegate

Single Program में एक user multiple delegate का करता है।

जैसे कि Program में एक से ज्यादा delegate बनाई multiple delegate होती।

i.e. `Public delegates void Calculation (int a, int b);`

`Public delegate void Show ();`

जैसे इन दोनों delegates से मात्र 2 function को Point होती।

## Single Cast delegates

एक delegate को एक method को Point होगा

इसे single cast delegates कहेंगे

एक से ज्यादा method की reference यह नहीं देती।

They are derived from System.

Delegates class

# Multicast Delegate

(१) ई delegate multiple method to Point ~~होती~~ multicast delegate ~~अंशगति~~ /

२। ई ~~द्वारा~~ ~~एक~~ ~~युक्त~~ ई।

they are derived from

System.MulticastDelegate Class.



# Extension Method

They allow us to inject additional methods. WITHOUT Modifying, deriving the original class, struct or interface.

Class A  
5 method

यदि Class में कुछ पहले मौजूद  
एक लिस्ट 5 Method है।  
तो इसे इस Class में कुछ Additional  
Method Add करें यह कर सकते हैं।  
इस Class की Source Code, or  
Permission नहीं है कि यह  
कि Extension Method use कर सकते हैं।

यदि हमें Inheritance से कर सकते हैं।  
यदि Inheritance इसका भी फायदा है।  
यदि Class A में ~~कुछ~~ ~~method~~  
~~कुछ~~ Class Sealed होता है।

तो इसका Class बनाने में Extension Method  
के लिए इसे इस Class static होता है।

Static Class B

Extension Method

```
{ public static void Func3( this Program P )
{
    Binding Parameter.
}
```

```
}
```

```
}
```

Program P = new Program();

P.Func3();

## ~~ANON~~ ANONYMOUS function

- Anonymous  $\lambda$  or  $\lambda$  or  $\lambda$  delegates  $\lambda$   
 $\lambda$  use  $\lambda$  /
- C# 2.0 introduced

Ex

```
public delegate void mydelegate (int a);
```

```
{
  :
}
```

```
static void main (string[] args)
```

```
{
```

```
    mydelegate obj = delegate (int a)
```

```
    {
```

```
        a = a + 10;
```

```
        Console.WriteLine(a);
```

```
    } // use Anonymous method
```

```
};
```

```
obj(5);
```

```
Console.ReadLine();
```

Output = 15



~~Serialisation~~

## Serialisation and Deserialisation

Serialisation:-

the process of convert object to string is serialisation.

Q

Deserialisation:- process of converting string to object is deserialisation.

# Garbage Collector

Memory Management Handling is done by G.C.

It is Perform 2 duties.

## ① Allocating the memory.

When New object is created by application G.C. will allocate the memory for that object with in managed heap.

## ② De-Allocating the memory

When An object is not used by the Application. G.C. will recognize it as unused object and G.C. will destroy unused object according to generation Algorithm.

Garbage Collector memory ko Manage करता है  
अगर Object ko memory से हटा देता है  
memory Release ko G.C. ko करता है।

ये Resource को हटा देता है। जो Information  
G.C. को पता नहीं चलता है वो Unmanaged Resources  
को कहते हैं जैसे ये Resources  $\Rightarrow$  File handling,  
Window handling, database Connection, network Sockets  
है। हमको अगर developer ko Program ko  
Unmanaged Resource ko हटा देना है। तो हमको ये को  
memory Release को को Code में करना पड़ेगा।



UNManaged - Resource  $\nrightarrow$  memory Release  
Dispose method USE  $\nrightarrow$  /

अनिकल 1 नप

# Structure

Struct Program

```
{  
    public void func1() { }  
}
```

- Structure belong to Procedural Language
- Class belong to Oop language
- Structures can contain methods, constructors, field, property, indexes etc.
- It is value type
- Stored in Stack memory.
- Stack memory also small in size.
- Heap " " is big in size



| Struct                                  | Class                         |
|-----------------------------------------|-------------------------------|
| ① value type                            | ① Ref type                    |
| ② Store in stack memory                 | ② stored in heap memory       |
| ③ belong to procedural language         | ③ belong to OOP language.     |
| ④ No need to new keyword to create obj. | ④ create obj with new keyword |
| ④ <del>int, float, double</del>         |                               |
| ⑤ String, obj                           | ⑤ int, float, double          |
| ⑥ there are no class                    | ⑥ Sealed, abstract class.     |
| ⑦ No inheritance                        | ⑦ inheritance Apply.          |

# Enum

19

Collection of constant

Enum gender

```
{  
    male  
    female  
    other  
}
```

Static void

```
{
```

Gender gender = Gender.male;

• ew (gender);

Constant like : days of week = 7 days

Month in year = 12 month

Temperature = high, low, medium



## Local And Global Variable

- Local Variable declare inside the function.
- Local Variable are created when the the function execution is start and lost when the function terminates.

### Global Variable

- Global variable declare outside a function.
- global Variable are created when the execution start and end with Program end.

## Linq

Linq in C# is use to work with data access from sources such as objects, data sets, SQL Servers and XML

To work with Linq query we need following things

① data source

② Query

Each query is a combination of

- initialization
- Condition
- Selection

there are 3 ways to write a query.

① Query Syntax ② Method Syntax ③ Mixed Syntax (Query + method).

Ex.

From object in datasource

where condition

Select object



OOPS

# Object Oriented Programming

## Four Pillars of oop

- ① Encapsulation      ③ Inheritance  
② Polymorphism      ④ ~~Pol~~ Abstraction

## Encapsulation

- It is mechanism of wrapping the data (Variable) and code acting on data (Method or Properties) together as single Unit

- Variable of a class can be hidden (Private access modifier) from other class, can access that variable only through the method or properties of current class, it is called data hiding.

8. Variable to Private Access modifier to through  
 hide ~~रखते~~ other class to ~~अगर~~ method to  
 public ~~आगे~~ / ~~नीचे~~ ~~होगा~~ user variable to  
~~अगर~~ Value ~~होगा~~ ~~अगर~~ ~~होगा~~

*[Signature]*

जो data हमें class में रखा है। उसको हमें Maximum private रखा है। Encapsulation हमें Security Layer Provide करता है।

## Example:-

### Class Program

{

private String Name;

private int Age;

} data

Public void setName (String Name)

{

this.Name = name;

}

Setter  
method

Public void getName ()

{

C.W ("Your name is:" + this.name

}

getter  
method

Code  
Acting  
ON  
data

Public void setAge (int Age)

{

this.Age = Age;

}

Public void getAge ()

{

C.W ("You age is:" + this.Age);

}

}

~~Program~~

Program p = new Program ();

p.setName ("");

p.setAge (-5);



उदाहरण P.set name की name और Age की -5

22

इसका है जो यह Encapsulation की समझ में आता है?

उदाहरण 1 में मैंने दिखाया है कि

जहाँ methods में Condition में आया है कि उन methods में Condition दिया

i.e

class Program {

private String name;

private int Age;

public void setName (String name)

→ Boolean  
value  
Return

{ if (String.IsNullOrEmpty (name)) == true)

{

C.W. ("Name is Required");

}

else

{

this.name = name;

}

}

public void getName (String.IsNullOrEmpty (name)) == true)

{

}

else { C.W. ("Your <sup>Name</sup> ~~age~~ is" + name);

Same. all. age function

if (age > 0)

{

this.age = Age

}

else

{

}

}

## Steps for encapsulation.

- ① Make fields or data or variable will be Private.
- ② Make Method will Public.
- ③ Make Method with conditions.

## BENEFITS

- A CLASS CAN HAVE TOTAL CONTROL OVER WHAT IS STORED IN IT'S FIELDS.
- THE FIELDS OF A CLASS CAN BE MADE READ-ONLY OR WRITE ONLY.



# POLYMORPHISM

• ONE thing Many forms.

• IT DERIVED FROM GREEK WORD "POLY" AND "MORPHS".

POLY Means = Many, MORPHS MEANS = FORMS  
Means MANY forms.

There are two types of Polymorphism.

① Static Polymorphism (Compile time Polymorphism)

② DYNAMIC Polymorphism (Run time Polymorphism)

## Static Polymorphism

Linking the function to the object at compile time so it's called Compile time polymorphism. Static polymorphism and also known as  
• Early binding.

We can achieve with

- function. OVERLOADING.
- OPERATOR "

## FUNCTION OVERLOADING

(एक) class के अंदर एक ही नाम के एक  
वर्गों function बना सकते हैं और  
(अनेक) Signature different-2 हैं।

## FUNCTION OVERLOADING :-

- Same Name Different Signature.

Ex.

```
Public void ADD ( )
```

```
{
```

```
    int a = 20;
```

```
    int b = 30
```

```
    int C = a + b;    (w. (C));
```

```
}
```

```
Public void ADD ( )
```

```
{
```

```
    String A = "ABC";
```

```
    String B = "DEF";
```

```
    String C = A + B;    (w. (C));
```

```
}
```

```
Public void ADD ( int A, int B )
```

```
{
```

```
    int C = A + B ;
```

```
    (w. (C));
```

```
}
```





# Dynamic Polymorphism

Linking function to the object at Run time is called Run-time Polymorphism, Late ~~by~~ binding.

We can achieve it by using of

① Method Overriding

## Method Overriding

- Same name Same Signature but different Implementation.
- Method overriding Allow us to have virtual and abstract method
- if <sup>define</sup> derived class; same method as defined in its base class, it is known as method overriding in C#.
- It is used to achieve Run time Polymorphism.

i.e

```

class Parent {
    public void Print()
    { c.w("Parent class method"); }
}

class Child : Parent {
    public void Print() { c.w("Child class"); }
}
    
```

अब अगर मैं child class का object बनाऊ तो child class का method चलेगा, वह अपने parent class के method को Hide कर लेगा।

method overriding करने के लिए अब मैं Parent class के method को virtual बनाऊंगा। और child class के method को override बनाऊंगा।

i.e

```
public class Parent {  
    public virtual void Print()  
    {  
        C.W("parent class method");  
    }  
}
```

```
class Child {  
    public override void Print()  
    {  
        C.W("child class method");  
        // base.Print() = parent class ki method  
        // चलेगा  
    }  
}
```

```
Parent p = new Child();  
p.Print();
```

O/P

Child class method

```
Child c = new Child();  
c.Print();
```

O/P

Child class  
method.



# Method hiding

जब Child class की Method, Parent class की Method की Hide करे तो Method hiding कहलाती है।

- ° A Method with Same Name and Same Signature as compare to other method with it belong to different (other) class.

```
Class Parent {
    Public void Show()
    { C.W("Parent class"); }
}
```

```
Class Child: Parent {
    Public new void Show()
    { C.W("Child class"); }
}
```

We use 'new' keyword in derived function.

\* \* 899 → other  
 \* \* 899 → 1. priority Parent → 2. new status

by default

## Access Specifiers

A. S

Internal

⇒

class

Public

⇒

Interface

Private

⇒

data member in class



# Inheritance

ONE CLASS PROPERTY IS INHERIT IN  
OTHER CLASS.

or

ONE CLASS IS DERIVED FROM ANOTHER  
CLASS CALLED INHERITANCE.

## Types of Inheritance

- Single Inheritance
- Hierarchical Inheritance.
- Multilevel Inheritance
- Multiple Inheritance.

## Single Inheritance

ONE CLASS IS DERIVED FROM OTHER  
BASE CLASS CALLED SINGLE INHERITANCE.

Class A

Class A

{

}



Class B

Class B: A

{

}

THE PROPERTY OF CLASS A IS ACCESSIBLE  
ON CLASS B.

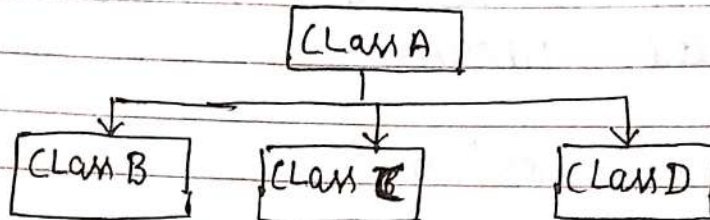
Class A is Parent class or Base class

Class B is Child class or derived class

- ONE Parent class and ONE child class

## Hierarchical Inheritance

- ONE Base class and multiple derived class.



Class A { }

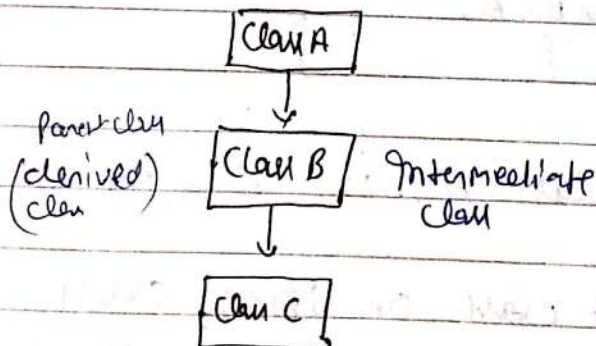
Class B : A { }

Class C : A { }

- ONE Parent class and multiple child class.

## Multilevel Inheritance

- A Parent class which is derive class.



- A Base class also child class.



```
Class A { }
```

```
Class B: A { //Intermediate class }
```

```
Class C: B { }
```

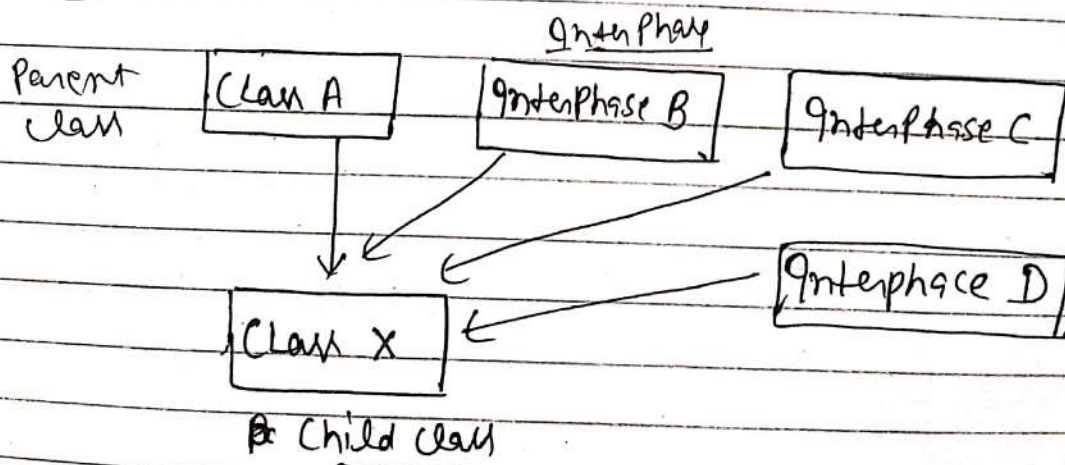
### MULTIPLE Inheritance with Interface

We can't create more than one Parent class. So we can use Interface.

एक class के 1 से ज्यादा Parent हो सकते हैं।

एक Parent class को एक से अधिक Interface का भी होना सके।

Ex.



Class A is Parent class

Interface B, C, D is Parent Interface.

Class X is child or derive class.

Class A { }

Interface B { }

Interface C { }

Interface D { }

Class X : A, B, C, D { }

Class X can Access the Property  
of Class A, Interface B, C, D,



## ABSTraction

The Process of hiding certain detail only show essential information is Abstraction.

i.e Console.WriteLine ( )

We can only use this for print but we can not know its internal code. So it's example of Abstraction.

i.e Class A {

Public int a;

Public int b;

Private void Addition ( )

{ int c = a + b;

C.W (c);

}

Private void Show()

{ this.Addition ( );

}

}

Abstraction is achieved by using Abstract class.

### Abstract Class

A class which contains the 'abstract' keyword.



• Abstract class में ~~object~~ object नहीं बनाया जा सकता।

abstract class Person

{

public abstract void show();

}

~~abs~~ class Student : Person

{

public override void show() {

}

class Teacher : Person

{

public override void show()

{

}

}



• Abstract class is Abstract method  
 ધોરણ છે, એક જોઈ શકે છે ધોરણ

Abstract Class Abstract method, Non-  
 Abstract method ધોરણ છે consist છે છે

• It is use every time as  
 a parent class.

• Can not declare abstract method  
 Outside of abstract class.

• Abstract class can not be declare  
 as sealed class.

• It can not be static.

• Multiple inheritance not support.

• Contain constructor or destructor.

Abstract method or method છે

આમાં body છે } એક છે  
 જોઈ Body child class છે

Override keyword use છે. નોંધ  
 Child class, abstract class નો method  
 નો Body છે

• An Abstract class can implement  
 property.

## Abstract class

- ① must be inherit
- ② Instance Not create
- ③ ~~Abstract class method~~  
~~abstract method child~~

④ override

⑤ contain Abstract and  
non Abstract method

⑥ abstract keyword use

## Sealed class

- ① can not be inherit
- ② Instance must be created.

③ Not override.

④ only Non-abstract  
method contain.

⑤ sealed keyword use.





# Interface

- Interface method is abstract method by default.

All Method inside the interface is by default abstract means

No Body of this method.

We can give body this method in it's child class.

i.e

```
Interface Employee
{
    void show();
}
```

```
class A : Employee
{
    public void show()
    {
        call ("Rund of Interface");
    }
}
```

Interface में सब चीजें (members) होते हैं  
by default public होते हैं।

हमें instance field और static field और चीजें  
हैं interface में सब चीजें बना सकते हैं।

```
Interface A {
    int A ; ❌ नहीं बना सकते ।
}
only method बना सकते हैं
```

## abstract class

- ① An abstract class can inherit a class and multiple interfaces.
- ② An Abstract class can have method with body.
- ③ ~~Implementing~~ override keyword is used during Implementation.
- ④ Can declare constructor or destructor.
- ⑤ Can have fields i.e. `int a, int b`
- ⑥ we can add or not Access modifiers.  
i.e. `public int a;`  
`int b;`
- ⑦ Access modifiers by default Private.
- ⑧ if you want to declare an abstract member in abstract class then abstract keyword is mandatory.

## Interface

- ① An interface can inherit multiple interface but not inherit a class.
- ② can not have method with body.
- ③ Not use override keyword during Implementation.
- ④ No constructor and no destructor.
- ⑤ Can't contain fields.
- ⑥ we can not add Access modifiers explicitly.
- ⑦ by default Public Access modifiers.
- ⑧ if we want to declare abstract member in interface then abstract keyword is not mandatory.



# Constructor

- magic method
- It is a special member function of a class because of the object is created it is automatically call.
- Constructor name is same as class name, and does not have Return type.

```
Class A
{
```

```
    public A() //Constructor
    {
    }
}
```

Constructor ~~execute~~ ~~first~~ ~~time~~ ~~execute~~ ~~first~~ |

## Types of Constructor

① default

② Parametrized

③ Copy Constructor

④ Private Constructor

⑤ Static "

## Default Constructor

Without Parameter Constructor is

Default Constructor. we can see it  
Previous page.

## Parameterized Constructor

A Constructor with Parameter,

Ex.

```
class A {
```

```
    int a,
```

```
    string b,
```

```
    int c
```

```
    public A (int a, string B, int C)
```

```
    { this.a = a; , this.c = C;
```

```
      this.b = B;
```

```
    }
```

Object Create like

```
A obj = new A(1, "A", 3);
```



# Copy Constructor

पारि Constructor वा (क) Object क Variable  
 कर Object क Copy करे  
 कर Object क रकने हे।

```
Class A {
```

```
    String name;
```

```
    int Age;
```

Parameters  
 ||

```
    Public A (String name, int age)
```

```
    {
```

```
    }
```

Copy  
 //

```
    Public A (A e)
```

```
    {
```

```
        this.name = e.name;
```

```
        this.Age = e.Age;
```

```
    }
```

```
A Obj = new Obj( "AB", 25);
```

```
Obj.getdata();
```

```
A Obj1 = new A (Obj);
```

Copy  
 constructor  
 call

## Private Constructor

अगर हम किसी class में Private Constructor बनाते हैं तो उस class की child class नहीं बना सकते।

और उस class में Object भी create नहीं कर सकते। और Inheritance भी नहीं होगा।

Class Example.

```
{
```

```
    Private Example()
```

```
{
```

```
}
```

```
Public Static void gettime ()
```

```
{
```

```
    Console.WriteLine(DateTime.Now);
```

```
}
```

```
Example.gettime();
```

Private Constructor जब भी बनाएंगे तो उसमें static member नहीं होगा।



## Static Constructor

Static Constructor, Static Variable or member or initialize karta hai and to perform Particular Action only ~~once~~ Once.

- Static Constructor is called only once. No matter how many object you create.

or (or or nahii)

- Static Constructor is called before instance (default or Parameterized) Constructor.

- A Static Constructor does not take any Access Specifier and No take any parameter.

Static Constructor or directly call hai idhar or karta / karta CLR or call karta hai

```
if class A
{
    public static String AB;
```

```
    static void A ()
    {
        AB = "AKA";
```

## Destructor

destructor is the special method which has same name as the class.

It is automatically invoked ~~the~~ when the program is completed or terming or ~~and~~ at the end of program.

It is start ~ before the class name and immediately de-allocate the memory of object.

Class A

{

~ A ()

{

}

}



## Static Class

- We can not create object.
- Can not inherit the class.
- Static keyword is use.
- It contain ONLY static data member and static method.
- Static constructor can create.

Static Class Product

```

{
    public static int Productid;
    public static String Productname;

    static Product()
    {
        Productid = 111;
        Productname = "Guites";
    }

    public static void getdetail()
    {
        c.w (Productid at Productname);
    }
}

Product.getdetail();

```

## Sealed class

- Sealed keyword is used
- Sealed class means that this class and its child class only can inherit it. Inheritance Apply only to sealed class.

Sealed class A

{

}

If Import method is in sealed class then both safety is there. If sealed class use then it is.

## Sealed method

Class A { public virtual void Print() { } }

Class B:A { public override void Print() { } }

Class C:A { public sealed override void Print() { } }

2nd Sealed method means that in class B the method is not used. Only class C can use the method.

Sealed method is for method overriding.



Pass by Value → Pass the actual value of the variable into the fun. (35)  
 Pass by Reference → Pass the variable itself into the function.

## Pass by Value & Pass by Reference

### Pass by Value

```

    public static void PassbyValue(int a)
    {
        a = a + 10;
        C.W(a);
    }
    
```

```

    static void main(String[] args)
    {
    
```

```

        int value = 5;
    
```

```

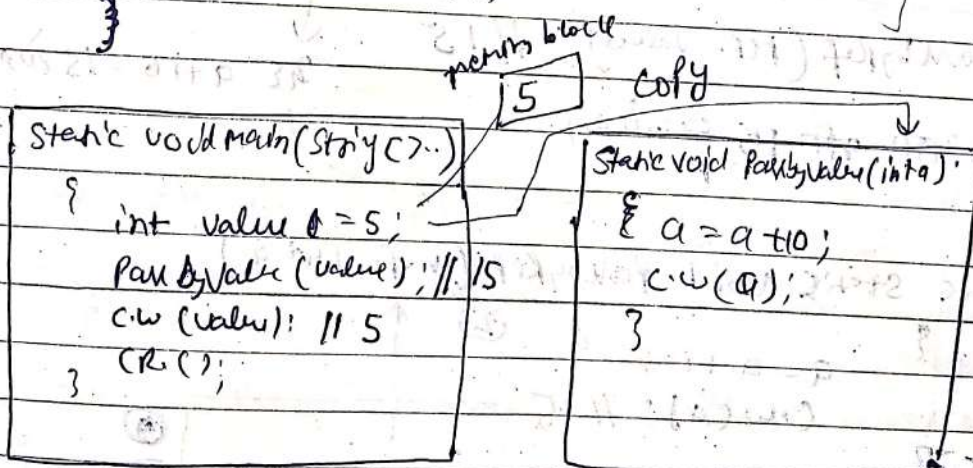
        PassbyValue(value);
    
```

```

        Console.WriteLine(value);
    
```

```

        Console.ReadLine();
    }
    
```



ये दोनों को एक ही memory block में है

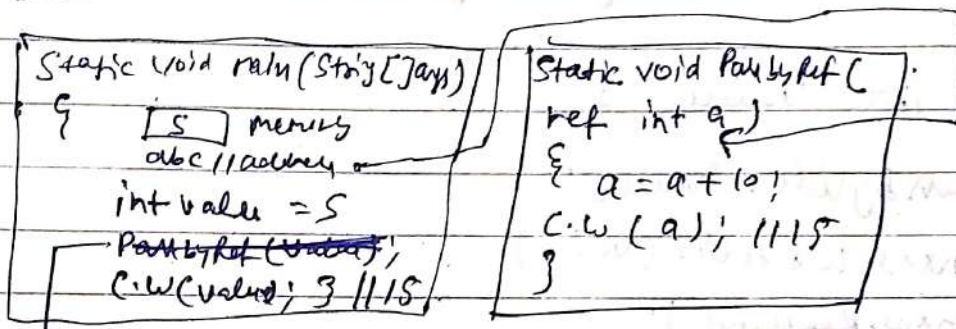
यह एक ही memory block में है

- value की एक copy ली जाती है। इसी function में value  $a + 10$  Add हो जाती है। इसका effect (value) 15 है।  
 यह value जो main memory block में है उसी value = 5 है। यह copy ली जाती है और Add होकर 15 हो जाता है।  
 value की copy travel होती है।

यहाँ पर भी memory block बनाता। जो address को  
Address होता। Address को हम Reference भी कहते हैं।

यहाँ पर हम Value copy करने नहीं करते, ~~क्यों~~ क्योंकि  
यहाँ memory block दोनो function में देते हैं,  
हमारा Reference है। जिससे कि अगर जो  
function में Value change होगी। जो वह  
Other function में भी change होगी।

Ex



PassbyRef (ref value); // 15

तो यहाँ भी 15 हो जाएगा।

यहाँ  $a + 10 = 15$  हो

Public Static void PassbyRef (ref int a)

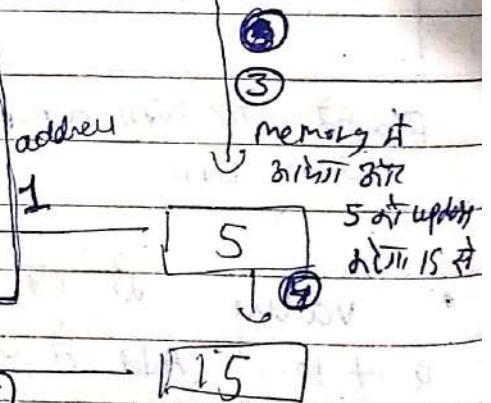
```

{
    a = a + 10;
    C.W(a); // 15
}
    
```

static void main (String[] args)

```

{
    int value = 5;
    PassbyRef (ref value);
    C.W (value); // 15
    C.R ();
}
    
```





# Pass by Out

21ET 4C 27 memory block same 27517

~~Static~~ Static void PanbyOut(out int a)

```
{
    a = 20;
    C.W(a); // 20
}
```

Static void main (String[] args)

```
{
    int value;
    PanbyOut(out value);
    C.W(value);
}
```

20 M.B

9

| Static void main (String[] args) | Static void PanbyOut (out int a) |
|----------------------------------|----------------------------------|
| {                                | {                                |
| int value;                       | a = 20;                          |
| PanbyOut (out value);            | C.W(a);                          |
| C.W (value); // 20               |                                  |
| }                                | }                                |

20 memory block

21ET value assign 27517 / 1000 function 27 400  
27517 27517 27517 value assign 27517 /

## Ref

- ① Called method and calling method must explicitly use the 'Ref' keyword before the value.
- ② the Parameter must be initialized first before it is passed to Ref
- ③ data can be passed bi-directionally.

## Out

- ① called method and calling method must explicitly use the 'Out' keyword before the value.
- ② It is not compulsory to initialize a Parameter before it passed to an out.
- ③ data is passed only in a unidirectionally (called method to caller method)

both they are treated differently at Run-time and treated same as compile time.

## Value type

- ① data will store in stack memory
- ② contain the actual data
- ③ structure, enum
- ④ int, long, char
- ⑤ hold the value.
- ⑥ faster in access
- ⑦ System.ValueType

## Reference type

- ① Heap memory.
- ② contain the Address of data.
- ③ class, interface, delegate.
- ④ string, obj.
- ⑤ hold a memory Address.
- ⑥ slower in access.
- ⑦ System.Object



## Class

- ① Reference type
- ② data store in Heap memory
- ③ Used to large Program
- ④ Limitless features
- ⑤ Contain Constructor and destructor
- ⑥ Use new keyword for create object
- ⑦ Inheritance perform
- ⑧ abstract, sealed class
- ⑨ belongs to OOPS language

## Struct

- ① Value type
- ② data store in Stack memory.
- ③ Used to Small Program
- ④ Limited feature.
- ⑤ Not contain
- ⑥ Create object with or without new keyword.
- ⑦ No inheritance.
- ⑧ No class contain
- ⑨ belongs to Procedural language

## type of()

- It is operator.
- It will return the type of given data type.

Ex: `CW(type of (int));`  
`CW(type of (byte));`

O/P  $\Rightarrow$  System.Int32  
 System.Byte.

- the operator takes the type itself as an argument.

## GetType()

- It is method.
- It will return the given variable data type.

Ex: `String S1 = "Hello";`  
`CW(S1.GetType());`  
 O/P  $\Rightarrow$  System.String

- It does not take any parameter.

## Parse

• Convert String to Any data type

• Null exception

## Convent

• Convert Any data type to Any data type

• Convert Null to zero

## dispose and Finalize

both are use to free unmanaged resources held by an object.

### Dispose ()

① define in IDisposable interface

② Need to be explicitly invoked by user.

③ Access Specifier is public

④ fast

### Finalize ()

① define in class

② invoked automatically by G.C.

③ Protected

④ Slower.

via ~~Destroy~~ Destructor compile it & it is not automatically  
finalize it convert it into it



## Var

- ① introduces in C# 3.0
- ② Variables are declared using var keyword
- ③ type of variable is decided by compiler at compile time
- ④ if the variable is not initialize, it throw an error.
- ⑤ Support intelligence in visual studio.
- ⑥ It can not be used in function
- ⑦ after assign the value could not change
- ⑧ Static type

## dynamic

- ① C# 4.0
- ② Variables are declared using dynamic keyword.
- ③ type of variable is decided by the compiler at run time.
- ④ if its variable is not initialize, it not throw an error.
- ⑤ Not Support.
- ⑥ It can be use for function.
- ⑦ can change
- ⑧ Dynamic type

## Stack

- ① It is a linear data structure.
- ② It access only local variable
- ③ It's variable can not be resize.
- ④ It's Allocation and de-Allocation are done by compiler
- ⑤ High Speed access
- ⑥ Faster

## Heap

- ① Hierarchical data structure.
- ② It access global variable.
- ③ Can be Resize.
- ④ allocation and deallocation are done by programmer.
- ⑤ Slower compared to stack.
- ⑥ Slower.

## Constant

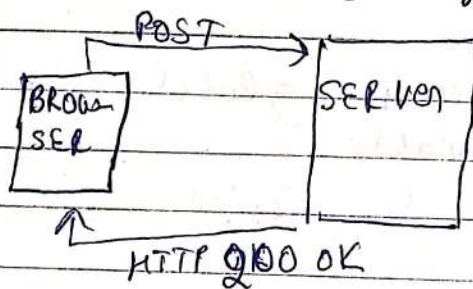
- ① It's value never be change.
- ② Can declare in method.  
(Fields)
- ③ Can not be used with static modifier.
- ④ ~~Const~~ field is compile Time Constant

## ReadOnly

- ① Change only in the public constructor not static or private constructor.
- ② It's field can not declare in method.
- ③ Can be used its fields with static modifier.
- ④ ReadOnly field can be used for run time constant.

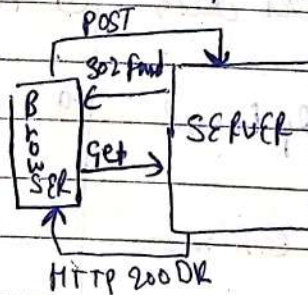
## Server Transfer

- ① Transfer is done by the server
- ② Browser URL <sup>does not</sup> changes ~~to Redirected target page.~~



## Response Redirect

- ① Transfer is done by the browser
- ② Browser URL ~~does not~~ change to Redirect target page.



both are use to transfer a user from one web page to another web page.



# ~~Partial Class~~

## Indexers

हमने Encapsulation use किया है उसमें हम check करेंगे कि किस तरह का data आया। और फिर उस इसके हम fields को Private रखें हैं और setter, getter method के throw check करें हैं कि क्या सही आये।

पर अगर यहाँ पूरी list encapsulate देंगे तो तो तो में Indexers use करेंगे हैं।  
Indexers is correct with help of this keyword.

Note: जो Access Modifiers में मानें हैं वह by default Private होगा class में।

i.e

class Employee

{

private int[] Age = new int[3];

public int this[int index]

{ set

{ Age[index] = value;

}

get

{

return Age[index];

} } }

class Program {

~~Static~~ static void main(String[] args)

{

21 zero index पर value set हो जायेगा

value at  
store index

```
Employee emp = new Employee();  
emp[0] = 5;
```

```
CW(emp[0]);
```

```
C.readLine();
```

कैसे बता रहे हैं की जगह -5 है तो यह Accept  
हो जायेगा क्योंकि इसकी Encapsulation को पूरा  
रखा नहीं गया है

कैसे  $emp[0]$  यहाँ zero रखे जाते हैं 1, 2,  
रख सकते हैं पर 3 से ज्यादा नहीं क्योंकि  
इसके जो Array बनाया है उसकी length 3 है

→ कैसे मैं यह दोनो काम करता हूँ।

```
Set { if (index >= 0 && index < Age.length)  
    if (value > 0)  
    {  
        Age[index] = value;  
    }  
}
```

```
else { CW("value is invalid"); }
```

```
}
```

```
else
```

```
{ CW("Invalid index");
```

```
}
```



## Partial Class

Partial Class is created by using Partial keyword.

एक class के code को file में रखना जब उस class को use करेंगे object बना देंगे तब उसके सब part combine होंगे और run होंगे।

ऐसी class जो parts में बनी है Partial Class कहलाती है।

# Couples

Couples show the relationship b/w modules in C#.

## Loosely Coupling

ONE object is not depend to other object is called loosely coupling.

Tight Coupling: group of classes are highly depends on another class.

Tight Coupling is not good, because it reduce the flexibility and re-usability of the code.

## Three Types of classes

- ① Client class :- Depend on the service class
- ② service class :- give service to client class.
- ③ Injector class :- inject the service class obj into client class.



## Boxing AND UNBOXING

Boxing :- Implicit Conversion of Value Type to Reference Type called Boxing.

int num = 3 // value type

Object obj = num // Reference Type

value Type = int, char, string, etc

UNBOXING :- Explicit Conversion of Reference type to value Type is called UNBOXING.

Object obj = num // Reference Type

int num1 = int(obj) // value type

# Dependency Injection

D.I is a software design Pattern that allow us to develop loosely Couple Code.

## Types of D. I.

### ① Constructor Injection:-

When we supply the ~~obj~~ Dependency Object through the constructor of class then it is called constructor injection.

### ② Property injection:-

When we supply the Dependency object through the public property of the class, then it is called as Property Injection.

### ③ Method Injection:-

When we supply the dependency object through the public method of the class then it is called Method Injection.



- A Collection is set of Related data That may not necessarily belong to the same data type.
- Other words "Collection is dynamic array".
- It's Length Can increase at Run-Time.
- Array Length is fix, but Collection Length is Auto Resize.
- Resize method of an array - destroy old array and create new array with new length

ie

```
int [] myarray = new int[3];
myarray[0] = 10;
myarray[1] = 20;
myarray[2] = 30;
Array.Resize(ref myarray, 4);
myarray[3] = 40;
```

2IFI Resize ki method 3 length ki array ko destroy krke New Array create krke 4 length ki

मान लीजिए कि एक Array बना है जिसमें Array में only same data type value है।  
 लेकिन इसे ऐसी Array बना है जिसमें Id, name, gender, birthdate, मोबाइल नं. आदि - ~~आदि~~ ~~आदि~~ Type की data है। तो हम Collection use करेंगे।  
 Collection में हम बनाएंगे - 2 तरह की data एक ~~आदि~~ ~~आदि~~ है और यह Auto resize होता है।  
 (dynamic length).

• we can't insert a ~~new~~ value into middle of an array because of its length

• we can't delete a value middle of array

### ARRAY

- Can't Resize at runtime
- Never Insert a value into middle of an array
- Never delete a value into middle of an array
- Same data type
- do not contain Any method for operation of an array.
- Fixed Length

### Collection

- Can Resize at run time.
- we can insert a value into middle of an array.
- we can delete a value into middle of an array.
- different - 2 data type.
- contain methods for operation on elements.
- Auto Resize.

We have two kind of Collection

#### ① Non-Generic Collection

- Stack, ArrayList, Hashtable, Sorted List etc.
- System.Collection have Non-generic Collection

#### ② generic Collection.

- List <T>, Linked List <T>, Queue <T>, Sorted List <T, V>
- System.Collection, generic namespace have generic collection.



generic

non-generic

① List <T> & LinkedList  
<T>

① ArrayList

② Dictionary <Key, Value>

② Hash Table

③ Queue <T>

③ Queue

④ Stack <T>

④ Stack

### Non-generic collection

ArrayList :- Collection of different - 2 data  
Type.

```
ArrayList an = new ArrayList();
```

```
an.add(10);
```

```
an.add("AB");
```

```
an.add(*10.5);
```

↓  
this length will  
be Auto length  
is

```
console.WriteLine(an.Capacity);
```

• 2nd (ArrayList) doubling its length automatically.

When we add value add more then its length 0 then 1

then New value add then length 2 then 3

then 2 more data add then its value will be 4

then 4 more value add then its length 4 then 5

then 5 more value add then its length 5 then 6

data add then its length 6 then 7

• Ex. If we create ArrayList an = new ArrayList(2);

then its length 2 it start from 0 to 1

3rd Collection of data is ArrayList  
in a data is print out of it  
foreach loop use

foreach (Object obj in arr)

↓  
list of type of  
data store in it  
is

{

Console.WriteLine(obj);

}

arr.Insert(indexNumber, value)  
(5, 25) ⇒ add 25 at 5 index  
arr.Remove(value) ⇒ remove 10 number

Console.WriteLine() Print out of it new line  
it will be

1

2

3

Console.WriteLine() line it print out of it horizontal,  
1 2 3

Non generic collection

→ ArrayList

→ Hash Table

→ Stack

→ Queue



## Hash Table

• Hash Table Store data in key/value formate.

• Array And Array List Store data in key value formate So what is the difference b/w them.

• IN Array and Arraylist Keys are predefined: Index Number.

We can't define keys explicitly in Array and Arraylist.

• अगर इंडेक्स में Index Number को According to the way it is in hash table use karli.

IN Hash Table:- We can give key explicitly

एक key को इंडेक्स Index Number define कर सकते हैं

{ अगर हम Array or Array List or Hash table use करें ?

Array or Arraylist

1  $\Rightarrow$  1123

2  $\Rightarrow$  Adil

3  $\Rightarrow$  Mera

4  $\Rightarrow$  25000

5  $\Rightarrow$  Hyderabad

अब अगर हम Salary को index में रखें तो Index Number काटेंगे।

1123 को 1123 में Index Number 'Salary' में रखें

इसलिए अगर हम इस Index Number 'Salary' में रखें

NT 2E ddt 2TS 2ET 2/2 Time 2/2 2/2

## Hash Table

id = 1123

name = Adil

designation = manager

Salary = 25000

City = Hyderabad.

## Example of Hash Table

```
HashTable HT = new Hashtable();
```

```
HT.Add("Name", "Adil");
```

```
HT.Add("Salary", 25000);
```

```
HT.Add("designation", "manager");
```

```
C.W(HT["Name"]);
```

or

```
foreach (Object Key in ht.Keys)
```

```
{
```

```
    Console.WriteLine(Key); // Key
```

```
    C.W(ht[Key]); // Value
```

Key & And Value.

```
    C.W(Key + " " + ht[Key]);
```

```
}
```



## Different Syntax of Hashtable

```
Hashtable ht = new Hashtable()
{
    { "Id", 1123 },
    { "Name", "Aadil" },
    { "Age", 27 }
};
```

ht.Remove("Salary");

Remove the item with specific key from the Hashtable. (यदि key को Remove कर देंगे)

Clean → Remove all item from hashtable.

ht.Clean();

## Get Hash Code

जब भी हम Hashtable में key define करेंगे तो Hash Code भी बनेगा। Hash Code को Integer Value होता है।

Hash table को बनाने के Algorithm implement होता होता है। यानी हम Hashing Algorithm है।

यदि Hashing Algorithm हर key को एक एक Random Hash code generate करता है।

जब भी Hashtable में key को search करेंगे तो Hashtable उस key को Hash code में

find करता है और उसे according view करता है।

यदि Hash code की value में Hashtable fast है Array or ArrayList से।

30th 30th Hash Code की बात है कि  
data receive होता है कि Random Retaining  
फंक्शन।

Property of Hash table

Count: यह Count बताता है कि Hash Table  
में कितने key value हैं।

Console.WriteLine(ht.Count);

30th 30th Hash Code बताता है कि यह

④ C.W("Adi", GetHashcode()); // Get hash code.

## Stack Non-Generic Collection

- ~~dup~~ duplication and Null Allow.

- Auto Resize.

- Not type Safe.

- Store homogeneous and Heterogeneous.

Not type Safe.

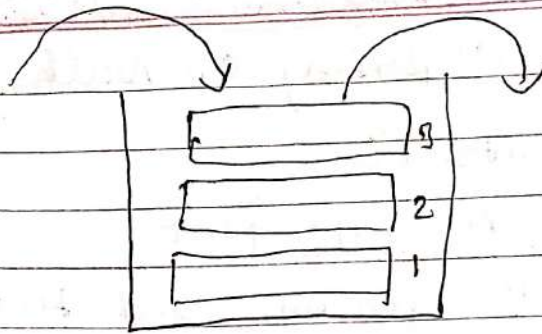
इसकी यह type के data store करती है।

Null ⇒ myStack.Push(Null)

duplication → myStack.Push("Ab");  
myStack.Push("Ab"); } → allow  
Duplication.



LIFO



Last in first out

Ex. Stack myStack = new Stack();

myStack.Push("Adil");

myStack.Push(27);

myStack.Push(5.5);

foreach (object item in myStack)

{  
    C.W(item);

}

|      |
|------|
| 5.5  |
| 27   |
| Adil |

Ex Stack में अगर Null और duplicate value  
present हो जाते हैं

USE Push method for Add Value. and Pop()  
Method or Peek() Retrive Value.

Property of Stack

Push → Add value

Count → Count value

Peek → Retrive value

Pop → Remove value

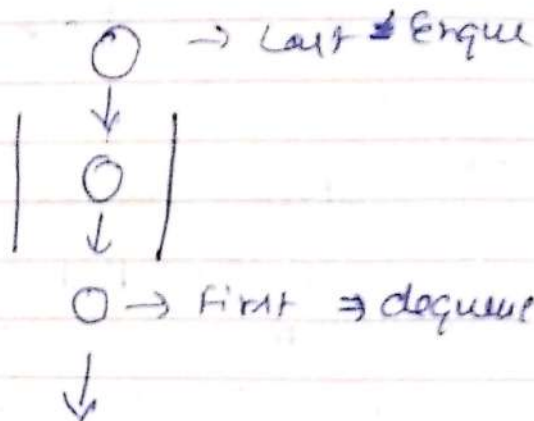
## Queue Non-generic

- duplicacy or Null Allow

- Auto Null
- Type safe Not
- Homogeneous and Heterogeneous.

F I F O

First in ~~out~~ first  
Out



Ex.

```
Queue myqueue = new Queue();
```

```
myqueue.Enqueue("Adil");
```

```
myqueue.Enqueue(25);
```

```
foreach (Object obj in myqueue)
```

```
{
    C.W(obj);
```

```
}
```

- Enqueue() method use to Add value.

- Dequeue() method use to Remove value

↳ वीत ऑर इटम क्वीम ऑर टॉप ऑर इटम  
उसकी Remove करेगा और उसी beginning ऑर

Item ऑर Remove ऑर करेगा ।

Peek() Return first item from the queue.

```
C.W(myqueue.Peek());
```

Namespaces

⇒ Dict

⇒ que

⇒ St

⇒ Lis

• they

गुण एव

que

ऑर

Type

<T



# Generic Collection

Namespace: - System.Collections.Generic

- ⇒ Dictionary
- ⇒ Queue < >
- ⇒ Stack < >
- ⇒ List

• They are type safe.

अगर हम generic Collection (List, Stack, Queue, Dictionary) का कोई Object बनाते हैं तो हम Object बनाते समय उसकी data-type specify करेंगे।

<T> ⇒ Data type specify करेंगे  
Like int, String

• Auto Resizing.

# Generic class

Generic class define functionality that can be used for any data type and are declare with a class declaration.

- C# 2.0 Introduce.

- Generic allow you to define a class with placeholder for the type of its field.

i.e

```
class Example <T>
```

```
{
```

```
    T box;
```

```
    public Example (T b)
```

```
    {
```

```
        this.box = b;
```

```
    }
```

```
    public T getBox()
```

```
    {
```

```
        return this.box
```

```
    }
```

```
}
```

~~Example~~ ~~=~~ ~~new Example~~

```
Example<int> q = new Example<int> (20
```

```
    C.W( q q.getBox());
```



# Singleton

the Singleton design Pattern ensures a class has only one instance and provide a global point of access it.

There are various way to implement Single-Ton pattern in C#.

- Private and Parameterless Single Constructor.
- Sealed class.
- Static variable to hold a Reference to the Single Created Instance.
- A Public and Static way of getting the Reference to the Created Instance.

## ADVANTAGES of Singleton

- ① it can implement interfaces.
- ② It help to hide dependencies.

Passing data from a Controller to view.

(51)

## Viewbag, ViewData, TempData

### Viewbag:

- transfer data from a Controller to view.
- it exists only for the current Request. and become null if the Request is Redirect.
- Viewbag is dynamic Property based on dynamic features
- Introduced in C# 4.0.
- it does not require typecasting when you use complex data type.  
(Automatic typecasting)

### Syntax:

Viewbag.<PropertyName> = <value>;

Viewbag.Var = "Hi, How are you";

Property: String value.

Value: it may be String, List, Object, array, int, char, float, double, DateTime etc;

### ViewData:

- Passes data from Controller to a View.
- ViewData is a dictionary, so it contains Key-Value Pair, where each key must be string.
- It is valid only during the current Request.

ViewData.Add("Id", 1);



be  
are

1-2

we can also pass the array in view def.

C# 2.

we can also pass the array in view def.

String[] fruits = { "Apple", "Banana", "Orange", "Grapes" };

```
viewData["FruitName"] = fruits;
```

iel

प्र Action method के view से दूसरे Action method के view में जाते data send होता जाता है।  
तो तब में view data use कर सकते हैं।

जब हम एक view में दूसरे view में viewdata का प्रयोग करने वाले भेजते हैं तो वह Null हो जायेगा।

इकाई ४ एम Tempdata use करेगा।

- View data Requires Type casting when you use complex data type to avoid error.

Syntax ViewData["key"] = <value>

Example

### Exam

Temp date

C.6 It is a dictionary object derived from the TempDataDictionary class.

- Tempdata store data as Key-value Pair.
- It is a temporary storage.

- 9th Val must be type last before

U.R. ( ~~स्वतंत्रता~~ must be ~~स्वतंत्रता~~ 75th ~~celebration~~ )

209 ए Type casting 



• OR Action method ke view se data  
Action method ke view se data nikal ke  
ke ke Tempdata mein rakhega  
ke ke Viewbag, ViewData mein rakhega

• Tempdata allows passing data from  
the current Request to the subsequent  
Request during Request redirection.

data ke view se ke view se nikal ke ke  
ke view se ke ke ke view se  
Tempdata Tempdata Null ke ke

MEAN Index view se About view se data  
nikal ke About view se Contact view  
se data ke ke

ke ke ke ke ke ke About view se

Contact view se ke data ke ke

About view se ke Tempdata Null ke ke

• (Subsequent Request se data Loss ke ke)

ke ke keep method use ke ke

ke ke About ke Action method se ke ke

Tempdata.Keep(); ke ke ke data

Loss ke ke

ke ke Contact se ke ke view se

ke data ke ke ke ke keep ke method

ke ke use ke ke ke ke ke ke ke

ke ke ke ke ke ke ke ke ke

ke ke ke ke ke ke ke ke ke ke



कैसे / कैसे - एक session + 467 है

## SESSION (mvc)

- Session का Property है `HttpServletRequest` class से derive होता है
- इसी mvc Application में multiple view में data Transfer हो रहा है
- default time is 20 minute. It can be increase.
- Session is valid for all Request.
- It also require type casting.

Syntax

```
Session["ABC"] = "HI, How are you?";
```

## ~~General~~ Routing in MVC

What is Routing?

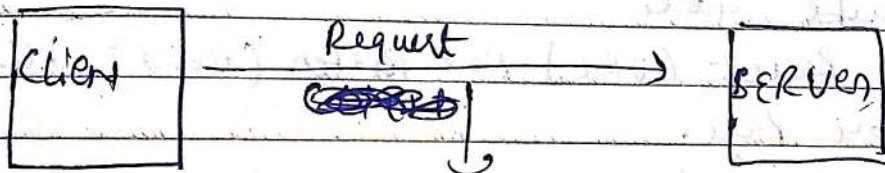
Routing is Process of mapping the Browser Request to the controller Action and Return response back.

- Routing maps incoming request (from browser) to a Particular Resources (Controller & Action Method)

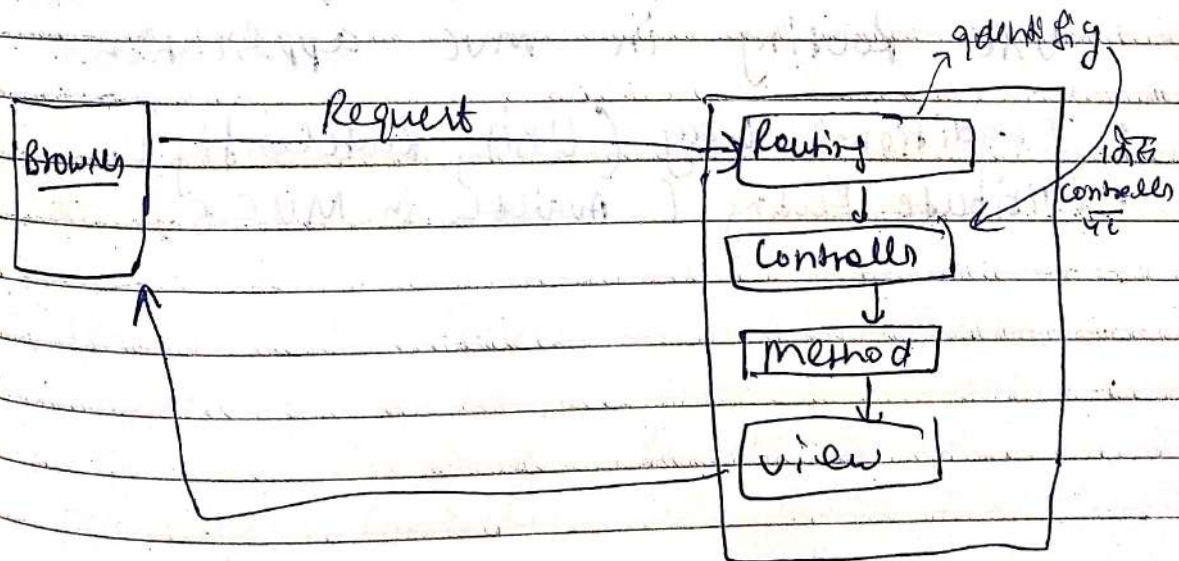
URL (Request) के बारे में URL + URL Response मिलती है  
 यह Routing है।

Routing map मिलती है कि किस URL पर

Controller के Action Method को Hit मिलती है।  
 यह Routing है।



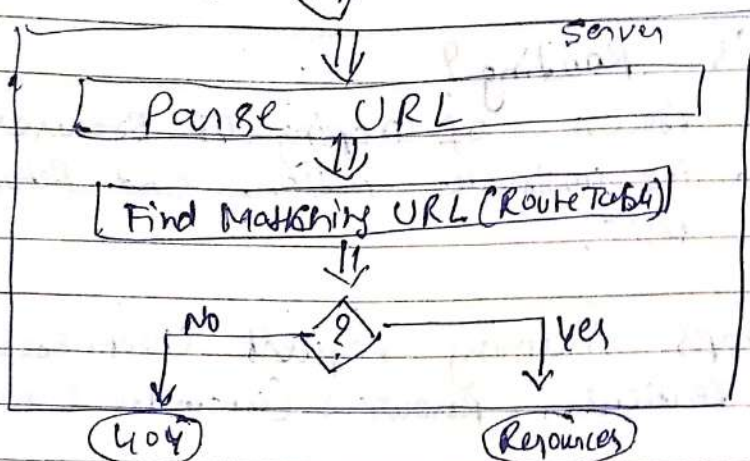
all about Request in Routing





Client

Request



## Route Table

- We define Route for each action method.
- All the routes are store in routing table.
- Each Incoming Request is mapped to this route table.
- if Request (URL) Not match (found) return 404 page.

There are two (2) ~~ways~~ type  
of routing in MVC application.

- Traditional way (Using RouteConfig).
- Attribute Routing (Available in MVC 5)



# Conventional or traditional Routing

## Route config →

- We Set all routes in RouteConfig file.
- Route Config is available in AppStart folder.

Route Config एक फाइल होगी जिसमें हम  
 (यहाँ) कुछ Pattern बना कर रख देंगे  
 जो यह URL को देखता तो यह Action Method चलेगा।

Route → It is a Pattern of Particular URL.  
 It needs following Parameters.

- Name
- Pattern
- Default values
- Constraints (if any)

Note: जब Application start होगी तो Route  
 Route Table में जोड़े पड़े चलेगा।

जो भी URL है वह Route बनाकर जोड़ी Register  
 करना पड़ता है। और यह Global.asax में

RouteConfig.RegisterRoutes(RouteTable.Routes); को  
 जोड़ें यह है Register को जोड़ें है।

- Conventional Routing is a Pattern  
 matching system for URL that  
 maps incoming Request to the  
 Particular Controller and action method.



## Attribute Routing

- MVC 5 Support it
- Attribute Routing uses attribute to define routes.
- It gives you more control over the URL in your web application

difference b/w XSS and SQL Injection

XSS is a Client Side vulnerability that Target other application users, while ~~SQL is server side~~

SQL Injection is server side vulnerability that targets other application database.

The main difference b/w XSS (Cross Site Scripting) and SQL injection is that

XSS inject malicious code to the website, therefore, that code is executed ~~that code is executed~~ in the view of the website, while SQL Injection injects the SQL code to a web form input field to obtain access and modify data.



# XSS

XSS  $\Rightarrow$  It is common website attacks. That is Capable of affecting the website as well as the users of the website. Attackers commonly use JavaScript to write malicious code in XSS.

The code can steal user's cookies, details, Change user settings, display various malware download and many more.

generally there are two types of XSS

- ① ~~Peris~~ persistent XSS
- ② Non - persistent XSS

① persistent XSS

the malicious is saved to the server in the database.

② Non - persistent XSS : the attacker sends the injected malicious code to the server through an HTTP request

# SOL Injection

(57)

It is a method of hacking a website. This method places a malicious code in the SOL Statment via webpage input.

Generally, a website collects the user details through a form. An attacker can provides an SOL Statment on the input fields instead of entering the real input values. This code can execute on the database of the website.